

National University of Computer and Emerging Sciences



AL-2002: Artificial Intelligence Lab Manual 14

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives	3
1 Introduction to CNNs	3
1.1 Input Layer	4
1.2 Convolution Layer	4
1.3 Activation Function (ReLU)	5
1.4 Pooling Layer	5
1.5 Flatten Layer	6
1.6 Fully Connected (Dense) Layer	6
2 Implementing CNN using Python	6

Objectives

After performing this lab, students shall be able to:

- ✓ Understand the working of Convolutional Neural Networks (CNNs).
- ✓ Identify and explain different CNN layers (Convolution, Pooling, Fully Connected).
- ✓ Implement a CNN model using Python and deep learning libraries.
- ✓ Train and evaluate CNN models on image datasets.
- ✓ Interpret results using accuracy and loss curves.

1 Introduction to CNNs

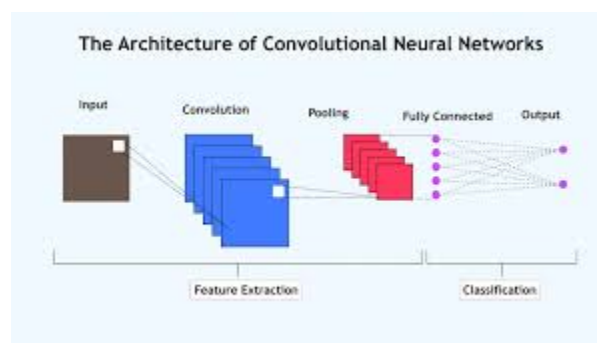
A Convolutional Neural Network (CNN) is a type of deep learning model primarily used for processing structured grid data such as images. CNNs automatically learn spatial hierarchies of features using convolution operations.

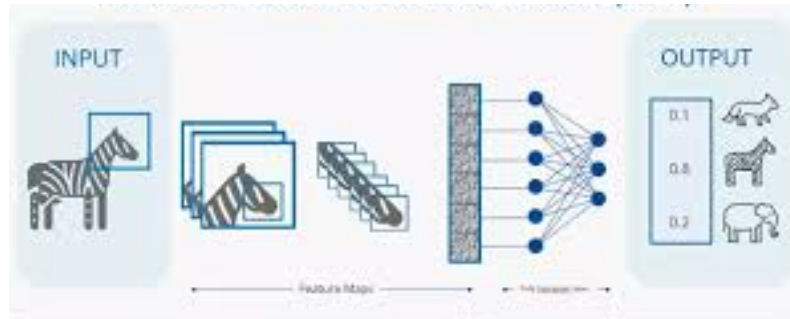
Unlike traditional neural networks, CNNs require less preprocessing and are highly effective in image classification, object detection, and pattern recognition tasks.

Images contain spatial information (height, width, channels). CNNs

- Preserve spatial relationships
- Require fewer parameters than fully connected networks
- Automatically extract features (edges, textures, shapes)

A typical CNN consists of multiple layers stacked together:





1.1 Input Layer

The input layer defines the shape of the data entering the network. For image data, this includes height, width, and number of channels. In PyTorch, the input shape is defined implicitly in the model. Images are passed in the format:

```
(batch_size, channels, height, width)
```

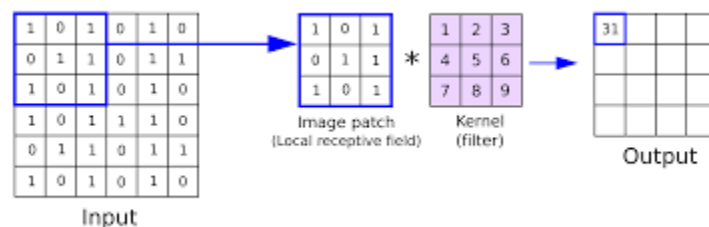
1.2 Convolution Layer

The convolution layer applies filters (kernels) to extract features from input images.

Mathematically:

$$\text{Feature Map} = \text{Input} * \text{Kernel}$$

Where: Input = Image matrix, Kernel = Filter matrix



```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3)
```

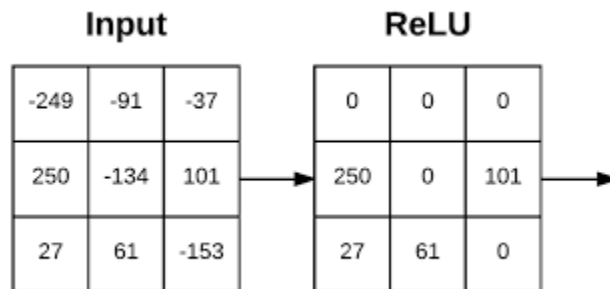
- **out_channels=32:** number of filters
- **kernel_size=3:** 3×3 filter

1.3 Activation Function (ReLU)

ReLU (Rectified Linear Unit) introduces non-linearity:

$$f(x) = \max(0, x)$$

It helps the network learn complex patterns.

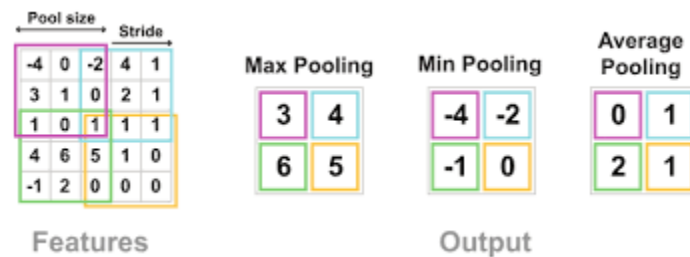


```
nn.ReLU()
```

1.4 Pooling Layer

Pooling reduces spatial dimensions and computation. Types:

- Max Pooling (most common)
- Average Pooling

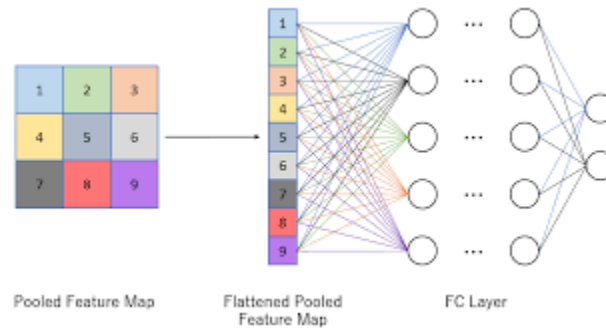


```
nn.MaxPool2d(kernel_size=2, stride=2)
```

1.5 Flatten Layer

Before passing data into dense layers, the 2D feature maps must be converted into a 1D vector.

```
torch.flatten(x, 1)
```



1.6 Fully Connected (Dense) Layer

Dense layers learn complex relationships between extracted features and output labels.

The final layer usually uses **softmax** activation for multi-class classification.

```
nn.Linear(in_features, out_features)
```

2 Implementing CNN using Python

We will use a sample dataset such as handwritten digits (MNIST).

First, the dataset (MNIST) is loaded, which contains grayscale images of handwritten digits. The pixel values are normalized by dividing by 255 so that all values lie between 0 and 1, improving training efficiency. The images are reshaped into a 4D format (samples, height, width, channels) because CNNs expect channel information.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms

# Transform: normalize to [0,1]
```

```

transform = transforms.Compose([
    transforms.ToTensor()
])

# Load dataset
train_dataset = datasets.MNIST(root='./data', train=True,
                                transform=transform, download=True)
test_dataset = datasets.MNIST(root='./data', train=False,
                                transform=transform)

# DataLoader
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64,
                                             shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=64,
                                           shuffle=False)

```

Next, a Sequential model is created, which allows stacking layers one after another. The first convolution layer applies 32 filters of size 3×3 to extract basic features from the image. This is followed by a max pooling layer that reduces the spatial dimensions, making computation efficient.

A second convolution layer with 64 filters is added to capture more complex patterns, followed again by pooling. The output is then flattened into a one-dimensional vector so it can be fed into dense layers.

A dense layer with 64 neurons learns high-level patterns, and the final dense layer with 10 neurons (one for each digit class) uses softmax activation to output probabilities.

```

class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()

        self.conv_layers = nn.Sequential(
            # First Conv Block
            nn.Conv2d(1, 32, 3),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            # Second Conv Block

```

```

        nn.Conv2d(32, 64, 3),
        nn.ReLU(),
        nn.MaxPool2d(2, 2)
    )

    self.fc_layers = nn.Sequential(
        nn.Flatten(),
        nn.Linear(64 * 5 * 5, 64), # adjust size after conv layers
        nn.ReLU(),
        nn.Linear(64, 10)
    )

    def forward(self, x):
        x = self.conv_layers(x)
        x = self.fc_layers(x)
        return x

model = CNN()

```

The model is compiled using the Adam optimizer and a suitable loss function for classification. It is then trained for multiple epochs, during which it learns patterns in the data.

```

criterion = nn.CrossEntropyLoss() # handles softmax internally
optimizer = optim.Adam(model.parameters(), lr=0.001)

epochs = 5

train_losses = []
train_accuracies = []

for epoch in range(epochs):
    running_loss = 0.0
    correct = 0
    total = 0

    model.train()

```

```

for images, labels in train_loader:
    optimizer.zero_grad()

    outputs = model(images)
    loss = criterion(outputs, labels)

    loss.backward()
    optimizer.step()

    running_loss += loss.item()

    _, predicted = torch.max(outputs, 1)
    total += labels.size(0)
    correct += (predicted == labels).sum().item()

accuracy = correct / total
train_losses.append(running_loss)
train_accuracies.append(accuracy)

print(f"Epoch {epoch+1}, Loss: {running_loss:.4f}, Accuracy:
{accuracy:.4f}")

```

Finally, the model is evaluated on test data to measure its accuracy (you can find other evaluation metrics as well in the same manner we covered under ANN).

```

model.eval()

correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

```

```
test_acc = correct / total  
print("Test Accuracy:", test_acc)
```

Visualization of Training

```
# Accuracy Plot  
plt.plot(train_accuracies, label='Train Accuracy')  
plt.legend()  
plt.title("Accuracy Curve")  
plt.show()  
  
# Loss Plot  
plt.plot(train_losses, label='Train Loss')  
plt.legend()  
plt.title("Loss Curve")  
plt.show()
```